CS-375 Operating Systems Lab Assignment: Thread-Safe Data Structures

Santiago Soto

October 16, 2026

Contents

1 Introduction **2**

2 Exercise 1: Thread-Safe Queue and Stack **2**

3 Exercise 3: Thread-Safe Priority Queue **8**

4 Exercise 4: Thread-Safe Circular Buffer **11**

5 Exercise 5: Thread-Safe Deque **16**

6 Exercise 6: Thread-Safe Linked List **20**

7 Conclusion **24**

1 Introduction

This report documents the implementations and analyses for the thread-safe data structures lab assignment, covering thread-safe queue and stack implementations, Producer-Consumer and Undo-Redo use cases, and additional exercises (priority queue, circular buffer, deque and linked list).

All programs were compiled with `g++ -o <name> <name>.cpp -pthread -std=c++11` on Ubuntu Linux. Every data structure follows the same principle: the underlying container is *private* and every public method acquires the object's `std::mutex` before touching it. Because the only way to reach the data is through a locked method, at most one thread can read or modify the container at any moment, which removes data races by construction.

All test programs also use a global `coutMutex` when printing. Without it, `std::cout` calls from different threads can interleave character-by-character and produce garbled lines. This lock is for readable output only; correctness of the data structures does not depend on it.

2 Exercise 1: Thread-Safe Queue and Stack

```
1  // CS375 - Lab 7 - Exercise 1
2  // Thread-Safe Queue and Stack with Producer-Consumer and Undo-Redo
3  //
4  // Compile: g++ -o thread_safe_data_structures
       thread_safe_data_structures.cpp -pthread -std=c++11
5  // Run:     ./thread_safe_data_structures
6
7  #include <iostream>
8  #include <queue>
9  #include <stack>
10 #include <mutex>
11 #include <thread>
12 #include <vector>
13 #include <string>
14 #include <chrono>
15 #include <atomic>
16
17 // Serializes console output so lines printed by different threads
       never interleave.
18 std::mutex coutMutex;
19
20 // Thread-safe Queue class
21 template <typename T>
22 class ThreadSafeQueue {
23 private:
24     std::queue<T> queue;
25     std::mutex mtx;
26
27 public:
28     ThreadSafeQueue() {}
29
30     void push(T value) {
31         std::lock_guard<std::mutex> lock(mtx);  // locked here,
               unlocked at end of scope
32         queue.push(value);
33     }
34
```

```
35      // Check-and-remove happens under a single lock, so two consumers
            can
36      // never both see the same front element.
37      bool pop(T& value) {
38          std::lock_guard<std::mutex> lock(mtx);
39          if (queue.empty()) return false;
40          value = queue.front();
41          queue.pop();
42          return true;
43      }
44
45      bool empty() {
46          std::lock_guard<std::mutex> lock(mtx);
47          return queue.empty();
48      }
49
50      size_t size() {
51          std::lock_guard<std::mutex> lock(mtx);
52          return queue.size();
53      }
54  };
55
56  // Thread-safe Stack class
57  template <typename T>
58  class ThreadSafeStack {
59  private:
60      std::stack<T> stack;
61      std::mutex mtx;
62
63  public:
64      ThreadSafeStack() {}
65
66      void push(T value) {
67          std::lock_guard<std::mutex> lock(mtx);
68          stack.push(value);
69      }
70
71      bool pop(T& value) {
72          std::lock_guard<std::mutex> lock(mtx);
73          if (stack.empty()) return false;
74          value = stack.top();
75          stack.pop();
76          return true;
77      }
78
79      bool empty() {
80          std::lock_guard<std::mutex> lock(mtx);
81          return stack.empty();
82      }
83
84      size_t size() {
85          std::lock_guard<std::mutex> lock(mtx);
86          return stack.size();
87      }
88  };
89
90  // Problem 1: Producer-Consumer Simulation
91  void producerConsumerProblem() {
```

```
92      ThreadSafeQueue<std::string> messageQueue;
93      const int NUM_PRODUCERS = 3;
94      const int NUM_CONSUMERS = 2;
95      const int MESSAGES_PER_PRODUCER = 5;
96      const int TOTAL_MESSAGES = NUM_PRODUCERS * MESSAGES_PER_PRODUCER;
97      std::atomic<int> messages_produced(0);
98      std::atomic<int> messages_consumed(0);
99      std::vector<std::thread> producers;
100     std::vector<std::thread> consumers;
101
102     auto producer = [&messageQueue, &messages_produced](int id) {
103         for (int i = 0; i < MESSAGES_PER_PRODUCER; ++i) {
104             std::string message = "Producer " + std::to_string(id) + "
                    Message " + std::to_string(i);
105             messageQueue.push(message);
106             messages_produced++;
107             {
108                 std::lock_guard<std::mutex> lock(coutMutex);
109                 std::cout << "Produced: " << message << std::endl;
110             }
111             std::this_thread::sleep_for(std::chrono::milliseconds(100))
                    ;
112         }
113     };
114
115     auto consumer = [&messageQueue, &messages_produced, &
            messages_consumed](int id) {
116         int consumed = 0;
117         while (true) {
118             std::string message;
119             // Read the counter BEFORE trying to pop. Producers
                    increment it only
120             // after pushing, so if it already equals TOTAL_MESSAGES
                    every message
121             // is in the queue; a failed pop afterwards means the queue
                     is truly
122             // drained. (Checking after the pop could miss a message
                    pushed in between.)
123             bool allProduced = (messages_produced == TOTAL_MESSAGES);
124             if (messageQueue.pop(message)) {
125                 consumed++;
126                 messages_consumed++;
127                 std::lock_guard<std::mutex> lock(coutMutex);
128                 std::cout << "Consumer " << id << " processed: " <<
                        message << std::endl;
129             } else if (allProduced) {
130                 break;  // All messages produced and queue empty
131             }
132             std::this_thread::sleep_for(std::chrono::milliseconds(150))
                    ;
133         }
134         std::lock_guard<std::mutex> lock(coutMutex);
135         std::cout << "Consumer " << id << " finished after processing "
                << consumed << " messages" << std::endl;
136     };
137
138     for (int i = 0; i < NUM_PRODUCERS; ++i) {
139         producers.emplace_back(producer, i);
```

```
    }
    for (int i = 0; i < NUM_CONSUMERS; ++i) {
        consumers.emplace_back(consumer, i);
    }

    for (auto& t : producers) t.join();
    for (auto& t : consumers) t.join();

    std::cout << "Summary: produced " << messages_produced << ", consumed " << messages_consumed
              << ", left in queue " << messageQueue.size() << std::endl;
}

// Problem 2: Undo-Redo System
void undoRedoProblem() {
    ThreadSafeStack<std::string> undoStack;
    ThreadSafeStack<std::string> redoStack;
    std::string currentText = "";

    // Each edit saves the old state on the undo stack and clears the redo
    // history (a new edit invalidates anything that could have been redone).
    auto editText = [&undoStack, &redoStack, &currentText](const std::string& newText) {
        undoStack.push(currentText);
        std::string temp;
        while (redoStack.pop(temp)) {
            // discard redo history
        }
        currentText = newText;
        std::cout << "Text updated to: " << currentText << std::endl;
    };

    auto undo = [&undoStack, &redoStack, &currentText]() {
        std::string prevText;
        if (undoStack.pop(prevText)) {
            redoStack.push(currentText);
            currentText = prevText;
            std::cout << "Undo - Current text: " << currentText << std::endl;
        } else {
            std::cout << "Undo - Nothing to undo" << std::endl;
        }
    };

    auto redo = [&undoStack, &redoStack, &currentText]() {
        std::string nextText;
        if (redoStack.pop(nextText)) {
            undoStack.push(currentText);
            currentText = nextText;
            std::cout << "Redo - Current text: " << currentText << std::endl;
        } else {
            std::cout << "Redo - Nothing to redo" << std::endl;
        }
    };
```

```
191
192     editText("Hello");
193     editText("Hello World");
194     editText("Hello Universe");
195     undo();
196     undo();
197     redo();
198     editText("Hello Galaxy");
199     redo();  // redo history was cleared by the edit above
200
201     std::cout << "Final text: " << currentText
202               << " (undo stack size: " << undoStack.size()
203               << ", redo stack size: " << redoStack.size() << ")" <<
                      std::endl;
204 }
205
206 int main() {
207     std::cout << "Problem 1: Producer-Consumer Simulation\n";
208     producerConsumerProblem();
209
210     std::cout << "\nProblem 2: Undo-Redo System\n";
211     undoRedoProblem();
212     return 0;
213 }
```

Explanation

How `std::lock_guard` ensures thread safety. `std::lock_guard<std::mutex> lock(mtx);` locks the mutex in its constructor and unlocks it in its destructor (RAII). The lock therefore covers exactly the scope of the method and is released on every exit path, including the early `return false` in `pop()` and any exception thrown by the container. The mutex cannot be left locked by mistake, which would deadlock every later caller.

In `ThreadSafeQueue` and `ThreadSafeStack`, `push`, `pop`, `empty` and `size` all lock the same mutex, so they are mutually exclusive. The key design decision is that `pop()` does the *check*, the *read* and the *removal* in one critical section:

```
1 bool pop(T& value) {
2     std::lock_guard<std::mutex> lock(mtx);
3     if (queue.empty()) return false;
4     value = queue.front();
5     queue.pop();
6     return true;
7 }
```

If a caller instead wrote `if (!q.empty()) { v = q.front(); q.pop(); }` using separately locked calls, another thread could empty the queue between the check and the read (a *check-then-act* race). Combining the steps makes the operation atomic.

Producer-Consumer. Three producers each push 5 messages into a shared `ThreadSafeQueue<std::string>`, and two consumers pop them. The counter `messages_produced` is a `std::atomic<int>`, so its increments from several producers are never lost.

Consumer termination condition. A consumer loops forever and stops only when `pop()` fails *and* all 15 messages had been produced. A failed `pop()` alone is not enough, because the queue may just be temporarily empty while producers are still running. Since each producer increments the counter *after* the push, a counter value of 15 means all 15 messages are already

in the queue. The order of the checks matters: the consumer reads the counter *before* calling `pop()`. If it checked afterwards, this interleaving would be possible: the pop fails, a producer pushes the last message and sets the counter to 15, then the consumer sees 15 and quits with one message still in the queue. Reading first guarantees that a failed pop really means the queue has been drained. (The original starter loop `while (consumed < 15)` counted per consumer, so with two consumers neither one would ever reach 15 by itself. The `break` was the only real exit, and I made that explicit.) The summary line confirms `produced 15, consumed 15, left in queue 0`.

Undo-Redo. `editText` pushes the current text onto the undo stack and clears the redo stack, because a new edit invalidates the redo history. `undo` pops the previous state and pushes the current state onto the redo stack. `redo` does the reverse. The sequence produces: Hello → Hello World → Hello Universe → (undo) Hello World → (undo) Hello → (redo) Hello World → Hello Galaxy. A final `redo` prints "Nothing to redo", which shows that the edit cleared the redo history.

Analysis

Output order varies between runs because the OS scheduler decides which thread runs next. That variation is expected. What never varies is that each message is consumed *exactly once*, and none are lost or duplicated.

If synchronization were removed: `std::queue` is not thread-safe. Two producers calling `push` simultaneously could both write to the same slot or corrupt the internal `std::deque` block pointers, which can lose messages or crash the program. Two consumers could both read the same `front()` before either calls `pop()`, so one message is processed twice and the next is popped without ever being read. If `messages_produced` were a plain `int`, the read-modify-write in `++` could lose increments. The counter might then never reach 15, and the consumers would spin forever.

Deadlock considerations: each method holds exactly one lock and never calls another locking method while holding it, so no lock-ordering cycle (and no deadlock) is possible. In the Undo-Redo system, each stack operation is individually atomic. The *combination* of `undoStack.pop` + `redoStack.push` + updating `currentText` is not atomic, however. That is acceptable here because the demo is single-threaded. With several editing threads, one extra mutex around each whole edit/undo/redo would be needed to keep the three pieces of state consistent.

Observed output (Figure 1). The three producers' messages interleave in a different order in each round (for example Producer 0, 2, 1 in round 1), which shows the scheduler running the threads truly concurrently. Consumer 0 processed 8 messages and Consumer 1 processed 7, so the work was shared between them, and the summary line `produced 15, consumed 15, left in queue 0` confirms that every message was consumed exactly once. Both consumers printed their "finished" line, which shows the termination condition works. In Problem 2, the stack states follow the expected sequence, and the final `Redo - Nothing to redo` shows that the "Hello Galaxy" edit cleared the redo history (final undo stack size 3, redo stack size 0).

Screenshot

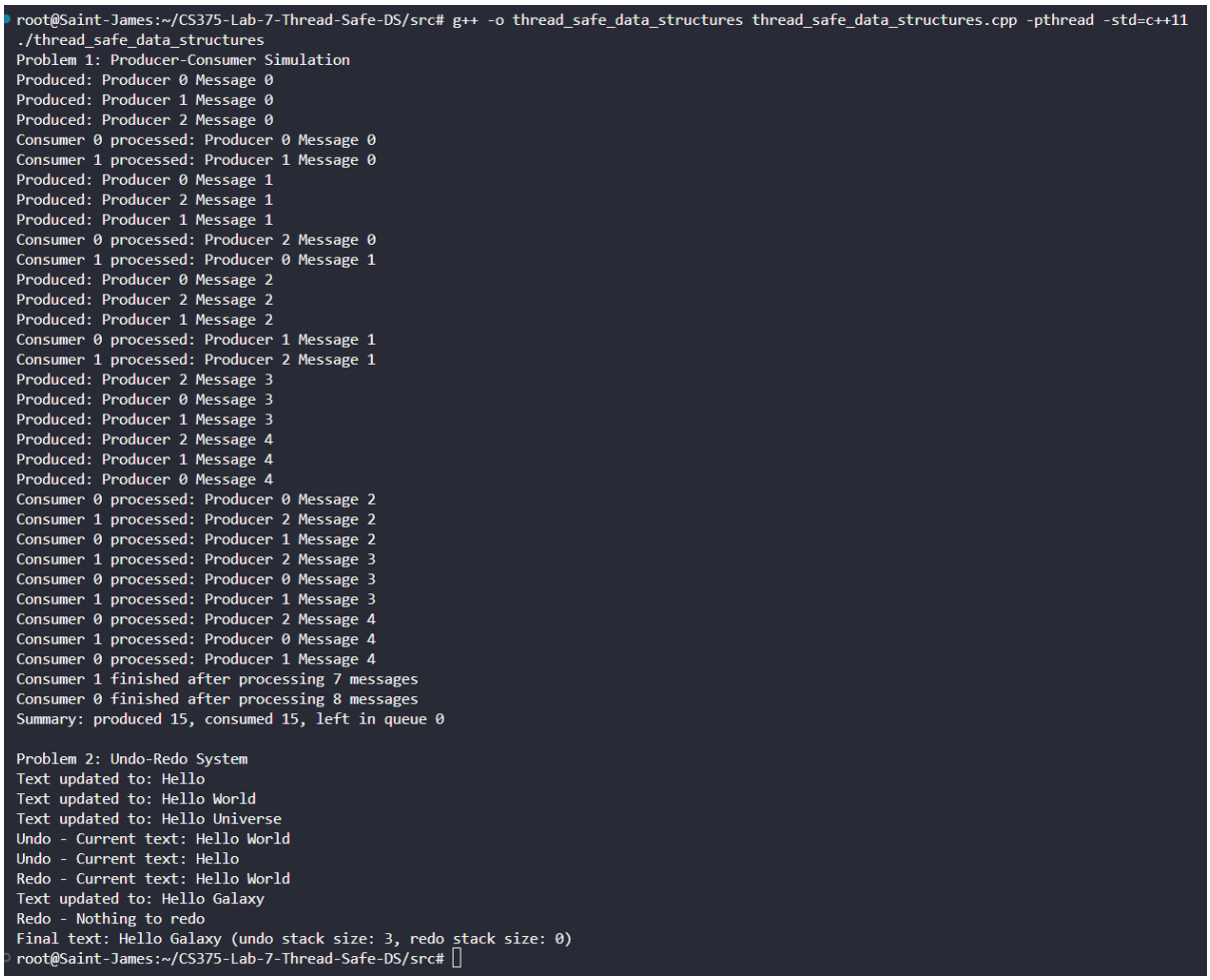


Figure 1: Compilation and execution of thread_safe_data_structures.cpp

3 Exercise 3: Thread-Safe Priority Queue

```cpp
// CS375 - Lab 7 - Exercise 3
// Thread-Safe Priority Queue
//
// Compile: g++ -o priority_queue priority_queue.cpp -pthread -std=c++11
// Run:     ./priority_queue

#include <iostream>
#include <queue>
#include <mutex>
#include <thread>
#include <vector>
#include <chrono>
#include <random>

// Serializes console output so lines printed by different threads never interleave.
std::mutex coutMutex;

```

```cpp
template <typename T>
class ThreadSafePriorityQueue {
private:
    std::priority_queue<T> pq;  // max-heap: top() is always the largest element
    std::mutex mtx;

public:
    void push(T value) {
        std::lock_guard<std::mutex> lock(mtx);
        pq.push(value);  // O(log n) sift-up restores the heap property
    }

    bool pop(T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        if (pq.empty()) return false;
        value = pq.top();
        pq.pop();        // O(log n) sift-down restores the heap property
        return true;
    }

    bool empty() {
        std::lock_guard<std::mutex> lock(mtx);
        return pq.empty();
    }

    size_t size() {
        std::lock_guard<std::mutex> lock(mtx);
        return pq.size();
    }
};

void priorityQueueTest() {
    ThreadSafePriorityQueue<int> pq;
    std::vector<std::thread> threads;
    const int NUM_THREADS = 4;
    const int PUSHES_PER_THREAD = 5;
    const int TOTAL = NUM_THREADS * PUSHES_PER_THREAD;

    // Each pusher gets its own random generator (rand() is not guaranteed thread-safe).
    auto pusher = [&pq](int id) {
        std::mt19937 gen(std::random_device{}() + id);
        std::uniform_int_distribution<int> dist(0, 99);
        for (int i = 0; i < PUSHES_PER_THREAD; ++i) {
            int priority = dist(gen);
            pq.push(priority);
            {
                std::lock_guard<std::mutex> lock(coutMutex);
                std::cout << "Thread " << id << " pushed: " << priority
                    << "\n";
            }
            std::this_thread::sleep_for(std::chrono::milliseconds(100));
        }
    };

```

```
71      // The popper keeps going until it has removed every pushed element
            .
72      auto popper = [&pq]() {
73          int popped = 0;
74          while (popped < TOTAL) {
75              int value;
76              if (pq.pop(value)) {
77                  popped++;
78                  std::lock_guard<std::mutex> lock(coutMutex);
79                  std::cout << "Popped: " << value << " (remaining: " <<
                        pq.size() << ")\n";
80              }
81              std::this_thread::sleep_for(std::chrono::milliseconds(150))
                    ;
82          }
83      };
84
85      for (int i = 0; i < NUM_THREADS; ++i) {
86          threads.emplace_back(pusher, i);
87      }
88      threads.emplace_back(popper);
89
90      for (auto& t : threads) t.join();
91
92      std::cout << "All " << TOTAL << " elements popped. Queue empty: "
93                << (pq.empty() ? "yes" : "no") << "\n";
94  }
95
96  int main() {
97      priorityQueueTest();
98      return 0;
99  }
```

Explanation

Maintaining order. `std::priority_queue<int>` is a binary max-heap stored in a `std::vector`. `push` appends the element and sifts it up, and `pop` moves the last element to the root and sifts it down. Both take $O(\log n)$ and preserve the heap invariant (each parent $\geq$ its children), so `top()` is always the largest element.

Thread safety. During a sift the heap is temporarily *invalid*. Elements are halfway through being swapped. If another thread read or modified the heap at that moment, it could observe or create a broken heap, after which `top()` would no longer be the maximum. Holding the mutex for the whole `push`/`pop` makes each heap update atomic, so other threads only ever see a valid heap. As in Exercise 1, `pop` combines `top()` and `pop()` under one lock.

I replaced `rand()` with a per-thread `std::mt19937` generator because `rand()` uses hidden global state and is not guaranteed to be thread-safe. The popper now loops until it has removed all 20 elements instead of making a fixed 20 attempts.

Analysis

Four pusher threads push 5 random priorities each (every 100 ms). One popper removes an element every 150 ms. While pushes are still arriving, the popper returns the largest element *currently* in the queue. A larger value pushed later can therefore appear after a smaller one was popped earlier, so the popped sequence is not globally sorted. Once all pushers finish (about 0.5 s), the remaining pops come out in strictly descending order, which demonstrates that the

heap order is preserved under concurrency. The final line confirms all 20 elements were popped and the queue is empty.

Because every operation takes the same lock, pushers and the popper are fully serialized on the queue. With heavy contention this lock becomes a bottleneck, but each critical section is only $O(\log n)$, so contention here is minimal.

Observed output (Figure 2). The first pop returned 88, but larger values (98, 93, 99) were pushed later. While pushers are still running, each pop returns the maximum of what is *currently* in the queue, not of everything that will ever be pushed. Once all 20 values were in the queue, the popper drained it in strictly descending order (99, 89, 88, 87, ..., 20, 20, 10), and the duplicate value 20 was handled correctly. The `remaining` counter drops by exactly one per pop down to 0, so no element was lost or duplicated.

Screenshot

```
root@Saint-James:~/CS375-Lab-7-Thread-Safe-DS/src# g++ -o priority_queue priority_queue.cpp -pthread -std=c++11
./priority_queue
Thread 0 pushed: 88
Thread 1 pushed: 77
Thread 2 pushed: 10
Popped: 88 (remaining: 3)
Thread 3 pushed: 82
Thread 1 pushed: 42
Thread 3 pushed: 45
Thread 2 pushed: 98
Thread 0 pushed: 25
Popped: 98 (remaining: 6)
Thread 2 pushed: 87
Thread 1 pushed: 88
Thread 0 pushed: 52
Thread 3 pushed: 93
Popped: 93 (remaining: 9)
Thread 1 pushed: 65
Thread 2 pushed: 20
Thread 0 pushed: 20
Thread 3 pushed: 43
Thread 1 pushed: 83
Thread 2 pushed: 89
Thread 3 pushed: 49
Thread 0 pushed: 99
Popped: 99 (remaining: 16)
Popped: 89 (remaining: 15)
Popped: 88 (remaining: 14)
Popped: 87 (remaining: 13)
Popped: 83 (remaining: 12)
Popped: 82 (remaining: 11)
Popped: 77 (remaining: 10)
Popped: 65 (remaining: 9)
Popped: 52 (remaining: 8)
Popped: 49 (remaining: 7)
Popped: 45 (remaining: 6)
Popped: 43 (remaining: 5)
Popped: 42 (remaining: 4)
Popped: 25 (remaining: 3)
Popped: 20 (remaining: 2)
Popped: 20 (remaining: 1)
Popped: 10 (remaining: 0)
All 20 elements popped. Queue empty: yes
root@Saint-James:~/CS375-Lab-7-Thread-Safe-DS/src#
```

Figure 2: Compilation and execution of priority_queue.cpp

4 Exercise 4: Thread-Safe Circular Buffer

```
1 // CS375 - Lab 7 - Exercise 4
```

```cpp
// Thread-Safe Circular (Bounded) Buffer
//
// Compile: g++ -o circular_buffer circular_buffer.cpp -pthread -std=c++11
// Run:     ./circular_buffer

#include <iostream>
#include <mutex>
#include <condition_variable>
#include <thread>
#include <vector>
#include <chrono>

#define BUFFER_SIZE 5

// Serializes console output so lines printed by different threads never interleave.
std::mutex coutMutex;

class ThreadSafeCircularBuffer {
private:
    int buffer[BUFFER_SIZE];          // fixed-size array, no dynamic allocation
    int in = 0, out = 0, count = 0;   // write index, read index, number of items
    std::mutex mtx;
    std::condition_variable not_full, not_empty;

public:
    // Blocks while the buffer is full.
    void push(int value) {
        std::unique_lock<std::mutex> lock(mtx);
        not_full.wait(lock, [this] { return count < BUFFER_SIZE; });
        buffer[in] = value;
        in = (in + 1) % BUFFER_SIZE;  // wrap around
        count++;
        lock.unlock();
        not_empty.notify_one();       // wake one waiting consumer
    }

    // Blocks while the buffer is empty.
    bool pop(int& value) {
        std::unique_lock<std::mutex> lock(mtx);
        not_empty.wait(lock, [this] { return count > 0; });
        value = buffer[out];
        out = (out + 1) % BUFFER_SIZE;  // wrap around
        count--;
        lock.unlock();
        not_full.notify_one();          // wake one waiting producer
        return true;
    }

    // Non-blocking variant: returns false immediately if empty.
    bool try_pop(int& value) {
        std::unique_lock<std::mutex> lock(mtx);
        if (count == 0) return false;
        value = buffer[out];
        out = (out + 1) % BUFFER_SIZE;
```

```
56          count--;
57          lock.unlock();
58          not_full.notify_one();
59          return true;
60      }
61
62      bool empty() {
63          std::lock_guard<std::mutex> lock(mtx);
64          return count == 0;
65      }
66
67      bool full() {
68          std::lock_guard<std::mutex> lock(mtx);
69          return count == BUFFER_SIZE;
70      }
71
72      int size() {
73          std::lock_guard<std::mutex> lock(mtx);
74          return count;
75      }
76  };
77
78  void circularBufferTest() {
79      ThreadSafeCircularBuffer cb;
80      std::vector<std::thread> producers, consumers;
81      const int NUM_PRODUCERS = 2;
82      const int NUM_CONSUMERS = 2;
83      const int NUM_ITEMS = 10;  // per producer; consumers split the
            total evenly
84
85      // Producers are faster than consumers, so the buffer fills up and
86      // producers block on not_full -- this demonstrates back-pressure.
87      auto producer = [&cb](int id) {
88          for (int i = 0; i < NUM_ITEMS; ++i) {
89              int value = id * 100 + i;
90              cb.push(value);
91              {
92                  std::lock_guard<std::mutex> lock(coutMutex);
93                  std::cout << "Producer " << id << " produced: " <<
                        value
94                            << "  [buffer " << cb.size() << "/" <<
                                BUFFER_SIZE
95                            << (cb.full() ? " FULL" : "") << "]\n";
96              }
97              std::this_thread::sleep_for(std::chrono::milliseconds(50));
98          }
99      };
100
101     auto consumer = [&cb](int id) {
102         const int toConsume = NUM_PRODUCERS * NUM_ITEMS / NUM_CONSUMERS
                ;
103         for (int i = 0; i < toConsume; ++i) {
104             int value;
105             if (cb.pop(value)) {
106                 std::lock_guard<std::mutex> lock(coutMutex);
107                 std::cout << "    Consumer " << id << " consumed: " <<
                        value
```

```
                            << "  [buffer " << cb.size() << "/" <<
                                BUFFER_SIZE << "]\n";
            }
            std::this_thread::sleep_for(std::chrono::milliseconds(150))
                ;
        }
    };

    for (int i = 1; i <= NUM_PRODUCERS; ++i) producers.emplace_back(
        producer, i);
    for (int i = 1; i <= NUM_CONSUMERS; ++i) consumers.emplace_back(
        consumer, i);

    for (auto& t : producers) t.join();
    for (auto& t : consumers) t.join();

    std::cout << "Done. Buffer empty: " << (cb.empty() ? "yes" : "no")
        << "\n";
}

int main() {
    circularBufferTest();
    return 0;
}
```

Explanation

The buffer is a fixed array of `BUFFER_SIZE = 5` integers with a write index `in`, a read index `out` and a `count`. Both indices advance with `(i + 1) % BUFFER_SIZE`, so they wrap around and reuse slots without any allocation.

Condition variables. Besides mutual exclusion, a bounded buffer needs *waiting*: a producer must wait while the buffer is full and a consumer must wait while it is empty. Two `std::condition_variable`s provide this:

- `not_full.wait(lock, [this]{ return count < BUFFER_SIZE; })` atomically releases the mutex and puts the producer to sleep. It re-acquires the lock when woken and returns only once the predicate is true.
- After inserting, the producer calls `not_empty.notify_one()` to wake one sleeping consumer. Consumers do the symmetric thing with `not_empty` / `not_full`.

`std::unique_lock` is required instead of `lock_guard` because `wait()` must unlock and relock the mutex. The predicate form of `wait` re-checks the condition in a loop, which handles *spurious wake-ups* and the case where another thread consumed the slot first. The lock is released before `notify_one()` so the woken thread does not immediately block on the mutex again.

Change from the starter code. In the starter code, `pop()` returned `false` on an empty buffer instead of waiting. The consumer's fixed 10-iteration loop could then finish having consumed fewer than 10 items. Items would be silently left in the buffer, and if more than 5 were left the producer would block on `not_full` forever (a hang). I made `pop()` wait on `not_empty`, as the requirement to use condition variables implies, and kept the non-blocking behaviour as a separate `try_pop()`. The test uses 2 producers and 2 consumers (10 items per producer, 10 per consumer), as required.

Analysis

Producers run every 50 ms and consumers every 150 ms, so the buffer fills to 5/5 (printed as `FULL`). Producers then block until a consumer frees a slot. This is *back-pressure*: the fast side is throttled to the speed of the slow side, and memory use stays bounded. All 20 items are consumed and the buffer ends empty.

Comparison with `ThreadSafeQueue` (Exercise 1):

	ThreadSafeQueue	**ThreadSafeCircularBuffer**
Capacity	Unbounded (grows with `std::queue`)	Fixed, 5 slots
Memory	Dynamic allocation on growth	Pre-allocated array, no allocation
Synchronization	`mutex` + `lock_guard` only	`mutex` + two condition variables
Empty pop	Returns `false` immediately; caller must poll (sleep and retry)	Blocks until data arrives, with no busy-waiting
Full push	Never full	Blocks until space is free
CPU use while idle	Polling wastes wake-ups	Sleeping threads use no CPU
Risk	Unbounded memory if producers outpace consumers	Deadlock if consumers stop while producers wait

Without the condition variables, a producer would have to spin while the buffer is full, wasting CPU. Without the mutex, two producers could write the same `buffer[in]` slot and both increment `count`, which loses one item and corrupts the count.

Observed output (Figure 3). From the moment `104` was produced, the buffer repeatedly reaches `5/5 FULL`. After that, every new item appears only right after a consumer frees a slot, which is the producers blocking on `not_full` as intended. All 20 values (100–109 and 200–209) are consumed exactly once, and the run ends with `Buffer empty: yes`.

Some printed occupancies look odd, for example `Producer 2 produced: 200 [buffer 0/5]` or `Consumer 2 consumed: 203 [buffer 5/5]`. This is not a bug in the buffer. The program calls `size()` *after* `push()`/`pop()` has released the lock, so other threads can run in between. In the first case a consumer had already removed item 200; in the second a producer had already refilled the slot. Each method is atomic on its own, but a *sequence* of calls is not, so any value read after the lock is released is only a snapshot. This is exactly the check-then-act issue discussed in Exercise 1.

Screenshot

```
root@Saint-James:~/CS375-Lab-7-Thread-Safe-DS/src# g++ -o circular_buffer circular_buffer.cpp -pthread -std=c++11
./circular_buffer
Producer 1 produced: 100  [buffer 2/5]
Producer 2 produced: 200  [buffer 0/5]
    Consumer 2 consumed: 100  [buffer 0/5]
    Consumer 1 consumed: 200  [buffer 0/5]
Producer 1 produced: 101  [buffer 1/5]
Producer 2 produced: 201  [buffer 2/5]
Producer 1 produced: 102  [buffer 3/5]
Producer 2 produced: 202  [buffer 4/5]
    Consumer 2 consumed: 101  [buffer 3/5]
    Consumer 1 consumed: 201  [buffer 2/5]
Producer 1 produced: 103  [buffer 3/5]
Producer 2 produced: 203  [buffer 4/5]
Producer 1 produced: 104  [buffer 5/5 FULL]
    Consumer 2 consumed: 102  [buffer 4/5]
    Consumer 1 consumed: 202  [buffer 3/5]
Producer 2 produced: 204  [buffer 5/5 FULL]
Producer 1 produced: 105  [buffer 5/5 FULL]
    Consumer 1 consumed: 103  [buffer 3/5]
Producer 1 produced: 106  [buffer 5/5 FULL]
    Consumer 2 consumed: 203  [buffer 5/5]
Producer 2 produced: 205  [buffer 5/5 FULL]
    Consumer 1 consumed: 104  [buffer 4/5]
Producer 1 produced: 107  [buffer 5/5 FULL]
    Consumer 2 consumed: 204  [buffer 4/5]
Producer 2 produced: 206  [buffer 5/5 FULL]
    Consumer 1 consumed: 105  [buffer 4/5]
    Consumer 2 consumed: 205  [buffer 4/5]
Producer 1 produced: 108  [buffer 5/5 FULL]
Producer 2 produced: 207  [buffer 5/5 FULL]
    Consumer 1 consumed: 106  [buffer 4/5]
Producer 2 produced: 208  [buffer 5/5 FULL]
    Consumer 2 consumed: 107  [buffer 4/5]
Producer 1 produced: 109  [buffer 5/5 FULL]
    Consumer 1 consumed: 206  [buffer 4/5]
    Consumer 2 consumed: 108  [buffer 3/5]
Producer 2 produced: 209  [buffer 4/5]
    Consumer 1 consumed: 207  [buffer 3/5]
    Consumer 2 consumed: 208  [buffer 2/5]
    Consumer 2 consumed: 209  [buffer 0/5]
    Consumer 1 consumed: 109  [buffer 0/5]
Done. Buffer empty: yes
root@Saint-James:~/CS375-Lab-7-Thread-Safe-DS/src#
```

Figure 3: Compilation and execution of circular_buffer.cpp

5 Exercise 5: Thread-Safe Deque

```cpp
// CS375 - Lab 7 - Exercise 5
// Thread-Safe Double-Ended Queue
//
// Compile: g++ -o thread_safe_deque thread_safe_deque.cpp -pthread -std=c++11
// Run:     ./thread_safe_deque

#include <iostream>
#include <deque>
#include <mutex>
#include <thread>
#include <vector>
#include <chrono>
#include <atomic>

```

```cpp
// Serializes console output so lines printed by different threads never interleave.
std::mutex coutMutex;

// One mutex guards BOTH ends. When the deque holds 0 or 1 elements the
// front and back are the same element, so separate per-end locks would race.
template <typename T>
class ThreadSafeDeque {
private:
    std::deque<T> deque;
    std::mutex mtx;

public:
    void push_front(T value) {
        std::lock_guard<std::mutex> lock(mtx);
        deque.push_front(value);
    }

    void push_back(T value) {
        std::lock_guard<std::mutex> lock(mtx);
        deque.push_back(value);
    }

    bool pop_front(T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        if (deque.empty()) return false;
        value = deque.front();
        deque.pop_front();
        return true;
    }

    bool pop_back(T& value) {
        std::lock_guard<std::mutex> lock(mtx);
        if (deque.empty()) return false;
        value = deque.back();
        deque.pop_back();
        return true;
    }

    bool empty() {
        std::lock_guard<std::mutex> lock(mtx);
        return deque.empty();
    }

    size_t size() {
        std::lock_guard<std::mutex> lock(mtx);
        return deque.size();
    }
};

void dequeTest() {
    ThreadSafeDeque<int> dq;
    std::vector<std::thread> threads;
    std::atomic<int> pushed(0), popped(0);

    auto push_front = [&dq, &pushed](int id) {
        for (int i = 0; i < 5; ++i) {
```

```
            dq.push_front(id * 100 + i);
            pushed++;
            {
                std::lock_guard<std::mutex> lock(coutMutex);
                std::cout << "Thread " << id << " pushed front: " << (
                    id * 100 + i) << "\n";
            }
            std::this_thread::sleep_for(std::chrono::milliseconds(100))
                ;
        }
    };

    auto push_back = [&dq, &pushed](int id) {
        for (int i = 0; i < 5; ++i) {
            dq.push_back(id * 100 + i);
            pushed++;
            {
                std::lock_guard<std::mutex> lock(coutMutex);
                std::cout << "Thread " << id << " pushed back: " << (id
                    * 100 + i) << "\n";
            }
            std::this_thread::sleep_for(std::chrono::milliseconds(100))
                ;
        }
    };

    auto pop_front = [&dq, &popped]() {
        for (int i = 0; i < 10; ++i) {
            int value;
            if (dq.pop_front(value)) {
                popped++;
                std::lock_guard<std::mutex> lock(coutMutex);
                std::cout << "    Popped front: " << value << "\n";
            }
            std::this_thread::sleep_for(std::chrono::milliseconds(150))
                ;
        }
    };

    auto pop_back = [&dq, &popped]() {
        for (int i = 0; i < 10; ++i) {
            int value;
            if (dq.pop_back(value)) {
                popped++;
                std::lock_guard<std::mutex> lock(coutMutex);
                std::cout << "    Popped back: " << value << "\n";
            }
            std::this_thread::sleep_for(std::chrono::milliseconds(150))
                ;
        }
    };

    threads.emplace_back(push_front, 1);
    threads.emplace_back(push_back, 2);
    threads.emplace_back(pop_front);
    threads.emplace_back(pop_back);

    for (auto& t : threads) t.join();
```

```
123
124     std::cout << "Summary: pushed " << pushed << ", popped " << popped
125               << ", remaining " << dq.size()
126               << (pushed == popped + (int)dq.size() ? "  -> consistent
                      (no lost/duplicated items)" : "  -> INCONSISTENT")
127               << "\n";
128 }
129
130 int main() {
131     dequeTest();
132     return 0;
133 }
```

Explanation

Challenges of synchronizing both ends. It is tempting to give the front and the back their own locks so a thread at one end does not block a thread at the other. The two ends are not independent, however:

- When the deque holds **one** element, `front()` and `back()` are the *same* element. A concurrent `pop_front` and `pop_back` could both see it and both remove it. The result is one value returned twice and an underflow of the container.
- When it is **empty**, a `push_front` and a `pop_back` race on the same element.
- `std::deque` keeps shared bookkeeping (block map, size). Either end can reallocate the block map, which invalidates the other end's internal pointers.

For these reasons the implementation uses **one mutex for the whole deque**. Each operation, including the `empty()` check plus the read and removal in the pops, is a single atomic critical section. Fine-grained per-end locking is possible only with a custom structure (for example a doubly linked list with sentinel nodes), and it still needs special handling when the size is 0 or 1, which adds complexity and room for deadlock if two locks are ever taken in different orders.

Analysis

Thread 1 pushes to the front, thread 2 pushes to the back, and two threads pop from the front and back. Elements from thread 1 accumulate at the front (most recent first) and elements from thread 2 at the back, so *Popped front* mostly returns 1xx values and *Popped back* mostly returns 2xx values. When the deque gets short, either popper can take an element pushed from the other end. The final summary checks that `pushed = popped + remaining`, so no element was lost or duplicated. Some pop attempts find the deque empty and simply return `false`.

Without the mutex, the size-0/1 races above would lead to double pops, lost elements or a crash. The cost of the single lock is that operations on opposite ends cannot run in parallel. For short critical sections like these, that cost is negligible compared to the safety gained.

Observed output (Figure 4). Front pops returned 1xx values and back pops returned 2xx values, as expected, since thread 1 only pushes at the front and thread 2 only at the back. One line appears out of order: `Popped back:  200` is printed before `Thread 2 pushed back: 200`. The push really did happen first (otherwise the pop could not have returned 200). The pushing thread released the deque lock and was then preempted before it acquired `coutMutex`, so the popper's message was printed first. Print order is therefore not the same as operation order. The summary `pushed 10, popped 10, remaining 0 -> consistent` confirms that no element was lost or duplicated.

Screenshot

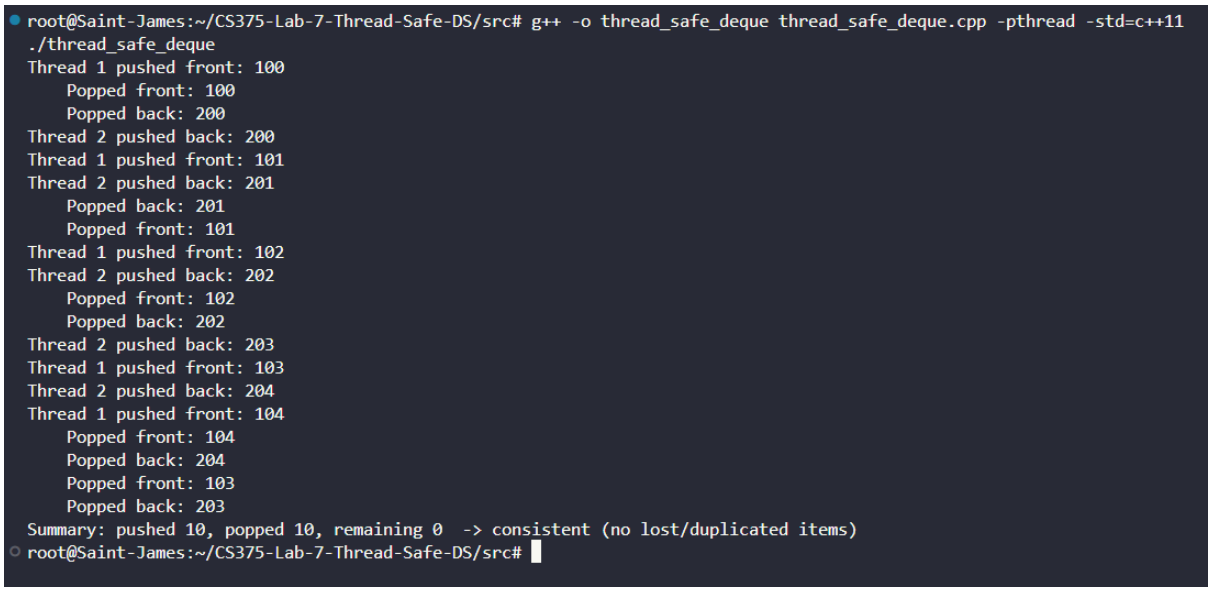


Figure 4: Compilation and execution of thread_safe_deque.cpp

6 Exercise 6: Thread-Safe Linked List

```cpp
// CS375 - Lab 7 - Exercise 6
// Thread-Safe Singly Linked List
//
// Compile: g++ -o thread_safe_linked_list thread_safe_linked_list.cpp -pthread -std=c++11
// Run:     ./thread_safe_linked_list

#include <iostream>
#include <mutex>
#include <thread>
#include <vector>
#include <chrono>

// Serializes console output so lines printed by different threads never interleave.
std::mutex coutMutex;

template <typename T>
class ThreadSafeLinkedList {
private:
    struct Node {
        T data;
        Node* next;
        Node(T value) : data(value), next(nullptr) {}
    };
    Node* head = nullptr;
    std::mutex mtx;  // coarse-grained: one lock protects the whole list

public:
    ThreadSafeLinkedList() {}
```

```
29      ThreadSafeLinkedList(const ThreadSafeLinkedList&) = delete;
                    // a copy would share nodes
30      ThreadSafeLinkedList& operator=(const ThreadSafeLinkedList&) =
            delete; // and double-delete them
31
32      ~ThreadSafeLinkedList() {
33          while (!empty()) {
34              T value;
35              pop_front(value);
36          }
37      }
38
39      // Reading head and writing head must happen atomically; otherwise
            two
40      // pushers could both link to the same old head and one node would
            be lost.
41      void push_front(T value) {
42          Node* newNode = new Node(value);  // allocate outside the lock
                (no shared state touched)
43          std::lock_guard<std::mutex> lock(mtx);
44          newNode->next = head;
45          head = newNode;
46      }
47
48      // Unlinking and deleting under the lock prevents two poppers from
49      // removing (and double-deleting) the same node.
50      bool pop_front(T& value) {
51          Node* temp;
52          {
53              std::lock_guard<std::mutex> lock(mtx);
54              if (!head) return false;
55              temp = head;
56              value = temp->data;
57              head = head->next;
58          }
59          delete temp;  // node is unreachable by other threads now
60          return true;
61      }
62
63      bool empty() {
64          std::lock_guard<std::mutex> lock(mtx);
65          return head == nullptr;
66      }
67
68      // O(n) traversal while holding the lock, so no node can be freed
            mid-walk.
69      size_t size() {
70          std::lock_guard<std::mutex> lock(mtx);
71          size_t count = 0;
72          Node* current = head;
73          while (current) {
74              count++;
75              current = current->next;
76          }
77          return count;
78      }
79  };
80
```

```cpp
void linkedListTest() {
    ThreadSafeLinkedList<int> list;
    std::vector<std::thread> threads;

    auto pusher = [&list](int id) {
        for (int i = 0; i < 5; ++i) {
            list.push_front(id * 100 + i);
            {
                std::lock_guard<std::mutex> lock(coutMutex);
                std::cout << "Thread " << id << " pushed: " << (id *
                    100 + i) << "\n";
            }
            std::this_thread::sleep_for(std::chrono::milliseconds(100))
                ;
        }
    };

    auto popper = [&list]() {
        for (int i = 0; i < 10; ++i) {
            int value;
            if (list.pop_front(value)) {
                std::lock_guard<std::mutex> lock(coutMutex);
                std::cout << "    Popped: " << value << "  (list size
                    now " << list.size() << ")\n";
            }
            std::this_thread::sleep_for(std::chrono::milliseconds(150))
                ;
        }
    };

    threads.emplace_back(pusher, 1);
    threads.emplace_back(pusher, 2);
    threads.emplace_back(popper);

    for (auto& t : threads) t.join();

    std::cout << "Final list size: " << list.size()
              << " (remaining nodes are freed by the destructor)\n";
}

int main() {
    linkedListTest();
    return 0;
}
```

Explanation

Maintaining list integrity. A singly linked list is defined entirely by its `head` pointer and the `next` pointers. Each operation modifies them in several steps, and the list must never be observed or modified between those steps:

- **Lost insert:** `push_front` reads `head` into `newNode->next` and then writes `head = newNode`. If two threads interleave, both new nodes point to the same old head, and the second write overwrites the first. One node becomes unreachable (lost data and a memory leak).
- **Double delete:** if two threads run `pop_front` simultaneously, both may read the same

`head`, return the same value and `delete` the same node twice. That is undefined behaviour, typically heap corruption or a crash.

- **Use after free:** `size()` walks the list. If a node is deleted while another thread is standing on it, the walker dereferences freed memory.

The mutex makes every read-modify-write of `head` atomic, and `size()` holds the lock for the whole traversal. Two small optimizations shorten the critical sections without breaking safety. `new Node` is allocated *before* taking the lock because it touches no shared state, and `delete temp` happens *after* releasing the lock because the node is already unlinked and no other thread can reach it. I also deleted the copy constructor and copy assignment, because a copied list would share and double-delete the same nodes. The destructor pops all remaining nodes so no memory leaks.

Analysis

Two pushers insert 5 nodes each at the head while a popper removes from the head. Since this is a LIFO structure, the popper usually gets the most recently pushed node. After pushing stops, nodes come out in reverse insertion order. The list size printed after each pop decreases consistently.

Performance considerations. This is *coarse-grained* locking: one mutex serializes all operations. It is simple and obviously correct. `push_front`/`pop_front` are $O(1)$, but `size()` is $O(n)$ and blocks every other thread for the whole traversal. Possible improvements:

- keep a `count` member updated under the lock, so `size()` is $O(1)$;
- *fine-grained (hand-over-hand) locking* with one mutex per node, which lets operations on different parts of the list proceed in parallel. It is more complex, and locks must always be acquired in list order to avoid deadlock;
- a *lock-free* stack using `std::atomic<Node*>` and compare-and-swap on `head`. This has no blocking, but it must deal with the ABA problem and safe memory reclamation.

Since all operations here act only on the head, fine-grained locking would gain nothing. A single mutex is the right choice for this workload.

Observed output (Figure 5). Because `push_front`/`pop_front` form a LIFO stack, each pop returns the most recently inserted node (for example 100 right after it was pushed). After both pushers finished, the remaining nodes came out in reverse insertion order (204, 104, 203, 202, 102, 201, 200). The reported list size decreases by one with each pop down to 0, and all 10 nodes were removed exactly once. The list was never corrupted (no lost node, no double free, no crash).

Screenshot



Figure 5: Compilation and execution of thread_safe_linked_list.cpp

7 Conclusion

All five data structures achieve thread safety the same way: private state, one mutex per object, and RAII locks (`std::lock_guard` / `std::unique_lock`) that cover each operation entirely, including compound *check-then-act* steps such as "if not empty, read and remove". The circular buffer adds condition variables so threads can *wait* for a state change instead of polling. Removing the locks would produce lost updates, duplicated or lost elements, corrupted containers, double frees and crashes. Keeping each critical section short and never holding two locks at once prevents deadlock.